

CAPÍTULO 4

ALGORITMOS Y ESTRUCTURAS DE CONTROL REPETITIVAS EN LENGUAJE C/C++



Margarita Zambrano, César Villacís, David Alvarado, Luis García, Kevin Topón,
Margarita Villacís, Sofía Bayona, Luis Pastor

Universidad de las Fuerzas Armadas – ESPE
Quito, Ecuador

Universidad Politécnica Salesiana – UPS
Quito, Ecuador

Universidad Rey Juan Carlos – URJC
Madrid, España

Contenido

○ **Bucles anidados**

- Problema 4.13: (Aplicación de bucles anidados para visualizar un triángulo rectángulo isósceles de asteriscos)
- Caso de Estudio 4.8: Visualización de un Rombo con Asteriscos 254

Bucles anidados

En el lenguaje C/C++ es posible anidar bucles, donde estos bucles anidados constan de un bucle externo con uno o más bucles internos (Joyanes Aguilar, L., 2008). Cada vez que se repite el bucle externo, los bucles internos se repiten y se ejecutan las sentencias de estos bucles. Se pueden combinar bucles for() entre sí, pero también se pueden combinar bucles for() con bucles while() y do-while() en diferente orden, por ejemplo:

Ejemplo 4.1: Combinación de un bucle do-while() anidado con un bucle for().

```
do{
    for()
    {
        sentencia1;
        sentencia2;
        ...
        sentencian;
    }
}while();
```

Ejemplo 4.2: Combinación de un bucle for() anidado con un bucle while().

```
for()
{
    while()
    {
        sentencia1;
        sentencia2;
        ...
        sentencian;
    }
}
```

Contenido

- Bucles anidados
- **Problema 4.13: (Aplicación de bucles anidados para visualizar un triángulo rectángulo isósceles de asteriscos)**
- Caso de Estudio 4.8: Visualización de un Rombo con Asteriscos 254

Problema 4.13: (Aplicación de bucles anidados para visualizar un triángulo rectángulo isósceles de asteriscos)

Escribir un programa que permita visualizar un triángulo rectángulo isósceles de asteriscos. Así, por ejemplo, si el valor del lado del triángulo es igual a 5, se obtendrá la siguiente figura:

```
*  
**  
***  
****  
*****
```

Problema 4.13: (Aplicación de bucles anidados para visualizar un triángulo rectángulo isósceles de asteriscos)

N°	Algoritmo en Pseudocódigo	Código en Lenguaje C/C++
1	entero : lado	<code>int lado;</code>
2	entero : f	<code>int f;</code>
3	entero : a	<code>int a;</code>
4	Leer el valor del lado	<code>cin >> lado;</code>
5	Para: 1. Inicializar el valor del contador de las filas 'f' en uno. 2. Mientras la condición (f <= lado) sea verdadera, ejecutar las sentencias del bucle externo. 3. Luego de cada ciclo incrementar el valor del contador de las filas 'f' en uno.	<code>for (f = 1; f <= lado; f++) {</code>
6	Para: 1. Inicializar el valor del contador de las columnas de asteriscos 'a' en uno.	<code>for (a = 1; a <= f; a++) {</code>

	2. Mientras la condición (a <= f) sea verdadera, ejecutar las sentencias del bucle interno. 3. Luego de cada ciclo incrementar el valor del contador de las columnas de asteriscos 'a' en uno.	
7	Imprimir un asterisco por columna.	<code>cout << "*";</code> <code>}</code>
8	Imprimir un salto de línea (INTRO).	<code>cout << endl;</code> <code>}</code>

```

// Librerías.
#include <iostream>
#include <cstdlib>
#include <cmath>

using namespace std;

// Función principal.
int main()
{
    // Declaración de variables.
    int lado; // Entrada: Tamaño del lado del triángulo.
    int f;    // Auxiliar: Número de filas.
    int a;    // Salida: Número de columnas de asteriscos.

    // Leer el valor del lado del triángulo.
    cout << "Ingrese el tamaño del triángulo rectángulo isósceles: ";
    cin >> lado;

    // Bucle externo for()
    // Para:
    // 1. Inicializar el valor de las filas 'f' en uno.
    // 2. Mientras la condición (f <= lado) sea verdadera, ejecutar
    // las sentencias del bucle externo.
    // 3. Luego de cada ciclo incrementar el valor de las filas 'f' en uno.
    for (f = 1; f <= lado; f++)
    {
        // Bucle interno for()
        // Para:
        // 1. Inicializar el valor de las columnas de asteriscos 'a' en uno.
        // 2. Mientras la condición (a <= f) sea verdadera, ejecutar
        // las sentencias del bucle interno.
        // 3. Luego de cada ciclo incrementar el valor de las columnas de
        // asteriscos 'a' en uno.
        for (a = 1; a <= f; a++)
        {
            // Imprimir un asterisco por columna.
            cout << "**";
        }
        // Imprimir un salto de línea (INTRO).
        cout << endl;
    }

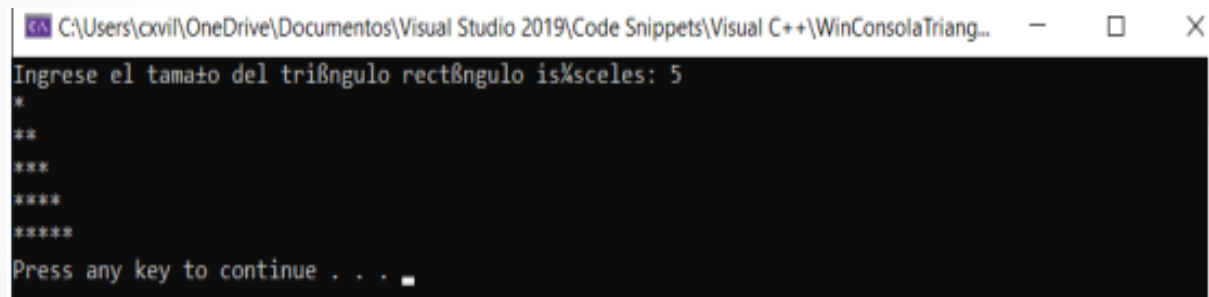
    // Incorporar una pausa en el programa.
    system("pause");
}

```

Programa 4.13. Código del Programa. Fuente:
Zambrano M., et al., 2025.

Problema 4.13: (Aplicación de bucles anidados para visualizar un triángulo rectángulo isósceles de asteriscos)

En la salida de la Ejecución 4.13 se puede ver la impresión del triángulo rectángulo isósceles de asteriscos de lado igual a 5



```
C:\Users\civil\OneDrive\Documentos\Visual Studio 2019\Code Snippets\Visual C++\WinConsolaTriang...
Ingrese el tamaño del triángulo rectángulo isósceles: 5
*
**
***
****
*****
Press any key to continue . . .
```

Ejecución 4.13. Ejecución del Programa . Fuente: Zambrano M., et al., 2025.

Problema 4.13: (Aplicación de bucles anidados para visualizar un triángulo rectángulo isósceles de asteriscos)

En la Tabla 4.13 se puede ver la prueba de escritorio del programa, donde la impresión del triángulo rectángulo isósceles se realiza mediante un bucle externo y un bucle interno. Cada vez que se repite el bucle externo, se ejecuta el bucle interno. El bucle externo se repite cinco veces en este caso, según la condición que depende del valor del lado del triángulo rectángulo y se controla con el contador de las filas 'f'. El número de repeticiones realizadas por el bucle interno se controla con el contador de los asteriscos 'a' cuya condición depende del valor de las filas 'f' del triángulo rectángulo, donde este bucle se encarga de manejar la impresión de los asteriscos.

Problema 4.13: (Aplicación de bucles anidados para visualizar un triángulo rectángulo isósceles de asteriscos)

lado	f	f<=lado	a	a<=f	cout << "***";	cout << endl;
5	1	1<=5(V)	1	1<=1(V)	*	
			2	2<=1(F)		INTRO
	2	2<=5(V)	1	1<=2(V)	**	
			2	2<=2(V)		
			3	3<=2(F)		INTRO
	3	3<=5(V)	1	1<=3(V)	***	
			2	2<=3(V)		
			3	3<=3(V)		
			4	4<=3(F)		INTRO
	4	4<=5(V)	1	1<=4(V)	****	
			2	2<=4(V)		
			3	3<=4(V)		
			4	4<=4(V)		
			5	5<=4(F)		INTRO
	5	5<=5(V)	1	1<=5(V)	*****	
			2	2<=5(V)		
			3	3<=5(V)		
			4	4<=5(V)		
			5	5<=5(V)		
			6	6<=5(F)		INTRO
	6	6<=5(F)				

Tabla 4.13. Prueba de Escritorio . Fuente: Zambrano M., et al., 2025.

Contenido

- Bucles anidados
- Problema 4.13: (Aplicación de bucles anidados para visualizar un triángulo rectángulo isósceles de asteriscos)
- **Caso de Estudio 4.8: Visualización de un Rombo con Asteriscos 254**

Caso de Estudio 4.8:

Visualización de un Rombo con Asteriscos 254

A) Problema

Escribir un programa que permita visualizar un rombo de asteriscos. Así, por ejemplo, si el valor de la Diagonal mayor es igual a 9 o a 10, se obtendrá la siguiente figura:

```
bbbb*  
bbb***  
bb*****  
b*****  
*****  
b*****  
bb*****  
bbb***  
bbbb*
```

Caso de Estudio 4.8:

Visualización de un Rombo

con Asteriscos 254

B) Análisis

Claramente se puede ver que la única entrada del problema es el valor de la diagonal mayor del rombo. Se requieren dos salidas que son los espacios en blanco y los asteriscos de la figura geométrica controlados por los contadores 'b' y 'a' respectivamente. Se requiere una variable auxiliar llamada 'aux' que tiene el valor de la mitad de la diagonal del rombo. El proceso de cálculo y visualización del rombo con asteriscos requiere la utilización de bucles anidados.

B.1) Requerimiento de los Datos

Entradas del Problema

diagonalMayor /* Tamaño de la diagonal mayor del rombo */

Salidas del Problema

Mensaje1 /* Mensaje que imprime un espacio en blanco */
Mensaje2 /* Mensaje que imprime un asterisco */
Mensaje3 /* Mensaje que imprime un INTRO */

Auxiliares del Problema

altura /* Altura que representa la mitad de la diagonal mayor del rombo */
f /* Número de filas */
b /* Contador para controlar el número de espacios en blanco */
a /* Contador para controlar el número de asteriscos */

Fórmulas Relevantes

$num = num \% 2$	(1)	/* Fórmula que calcula si un número es par */
$num = (num + 1) / 2$	(2)	/* Fórmula que obtiene la mitad de un valor impar y lo convierte a un valor par */
$num = num / 2$	(3)	/* Fórmula que obtiene la mitad de un valor par */
$b = h - f$	(4)	/* Fórmula que obtiene un número definido de espacios en blanco de acuerdo al valor de la altura del triángulo isósceles y al valor de la fila */
$a = 2 * f - 1$	(5)	/* Fórmula que obtiene un número impar de asteriscos de acuerdo al valor de la fila */

Caso de Estudio 4.8:

Visualización de un Rombo con Asteriscos 254

B.2) Diagrama de Entrada-Salida

En la Figura 4.8.1 se muestra el diagrama de Entrada-Salida del programa donde se identifican y se diagraman las entradas, salidas y auxiliares del problema como son: a) Entradas: la variable 'diagonalMayor'; b) Salidas: Mensaje1 que imprime un espacio en blanco; Mensaje2 que imprime un asterisco; Mensaje3 que imprime un INTRO; c) Auxiliares: la variable 'altura'; la variable 'f'; la variable 'b'; la variable 'a'.

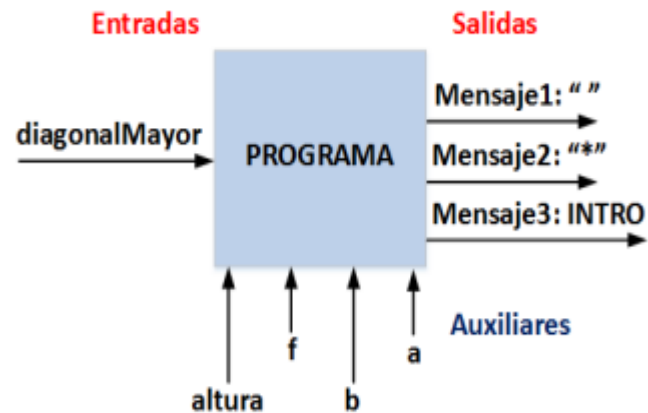


Figura 4.8.1 Diagrama de Entrada-Salida .
Fuente: Zambrano M., et al., 2025.

Caso de Estudio 4.8:

Visualización de un Rombo

con Asteriscos 254

C) Diseño

Una vez que se conoce las entradas, las auxiliares y las salidas del problema, se deben listar los pasos necesarios para resolver el problema, es decir, el algoritmo.

Algoritmo

1. Leer el valor de la diagonal mayor del rombo.
2. Si el valor de la diagonal mayor es impar, entonces
 - 2.1. Asignar a la variable 'altura' el valor de la diagonal mayor más uno dividido para dos.
3. Caso contrario,
 - 3.1. Asignar a la variable 'altura' el valor de la mitad de la diagonal mayor.
4. Para el bucle externo, hacer:
 - 4.1. Inicializar el valor del contador de las filas 'f' en uno.
 - 4.2. Mientras el valor de la condición ($f \leq \text{altura}$), sea verdadera, ejecutar las sentencias del bucle.
 - 4.3. Luego de cada iteración incrementar en uno el valor del contador de las filas 'f'.
 - 4.4. Para el bucle interno, hacer:
 - 4.4.1. Inicializar el valor de las columnas de blancos 'b' con el valor de la variable 'altura' menos el número de filas.
 - 4.4.2. Mientras la condición ($b \geq 1$) sea verdadera, ejecutar las sentencias del bucle interno.
 - 4.4.3. Luego de cada iteración decrementar el valor de las columnas de blancos 'b' en uno.
 - 4.4.4. Imprimir un espacio en blanco por columna.
 - 4.5. Para el bucle interno, hacer:
 - 4.5.1. Inicializar el valor de las columnas de asteriscos 'a' en uno.
 - 4.5.2. Mientras la condición ($a \leq 2 * f - 1$) sea verdadera, ejecutar las sentencias del bucle interno.
 - 4.5.3. Luego de cada iteración incrementar el valor de las columnas de asteriscos 'a' en uno.
 - 4.5.4. Imprimir un asterisco por columna.
 - 4.6. Imprimir un salto de línea (INTRO).

Caso de Estudio 4.8:

Visualización de un Rombo

con Asteriscos 254

C) Diseño

Una vez que se conoce las entradas, las auxiliares y las salidas del problema, se deben listar los pasos necesarios para resolver el problema, es decir, el algoritmo.

5. Para el bucle externo, hacer:
 - 5.1. Inicializar el valor de las filas 'f' con el valor de la variable 'altura' menos uno.
 - 5.2. Mientras la condición ($f \geq 1$) sea verdadera, ejecutar las sentencias del bucle externo.
 - 5.3. Luego de cada iteración decrementar en uno el valor del contador de las filas 'f'.
 - 5.4. Para el bucle interno, hacer:
 - 5.4.1. Inicializar el valor de las columnas de blancos 'b' con el valor de la variable 'altura' menos el número de filas.
 - 5.4.2. Mientras la condición ($b \geq 1$) sea verdadera, ejecutar las sentencias del bucle interno.
 - 5.4.3. Luego de cada iteración decrementar el valor de las columnas de blancos 'b' en uno.
 - 5.4.4. Imprimir un espacio en blanco por columna.
 - 5.5. Para el bucle interno, hacer:
 - 5.5.1. Inicializar el valor de las columnas de asteriscos 'a' en uno.
 - 5.5.2. Mientras la condición ($a \leq 2 * f - 1$) sea verdadera, ejecutar las sentencias del bucle interno.
 - 5.5.3. Luego de cada iteración incrementar el valor de las columnas de asteriscos 'a' en uno.
 - 5.5.4. Imprimir un asterisco por columna.
 - 5.6. Imprimir un salto de línea (INTRO).

Caso de Estudio 4.8:

Visualización de un Rombo con Asteriscos 254

D) Implementación

Para implementar la solución, se debe escribir el algoritmo como un programa de C/C++ que contenga toda la información necesaria para completar la traducción a lenguaje máquina. En la Tabla 4.8.1 se muestra el código del programa en C/C++.

```
/******  
WinConsolaCasoEstudio_4_8  
******/  
  
// Librerías.  
#include <iostream>  
#include <cstdlib>  
#include <cmath>  
  
using namespace std;  
  
// Función principal.  
int main()  
{  
    // Declaración de variables.  
    int diagonalMayor; // Entrada: Tamaño de la diagonal mayor del rombo.  
    int altura;        // Auxiliar: Altura que representa la mitad de la  
                        // diagonal mayor del rombo.  
    int f;              // Auxiliar: Número de filas.  
    int b;              // Salida: Número de columnas de espacios en blanco.  
    int a;              // Salida: Número de columnas de asteriscos.  
  
    // Leer el valor de la diagonal mayor del rombo.  
    cout << "Ingrese el valor de la diagonal mayor del rombo: ";  
    cin >> diagonalMayor;  
  
    // Si el valor de la diagonal mayor es impar  
    if (diagonalMayor % 2 != 0)  
    {  
        // Asignar a la variable 'altura' el valor de la diagonal mayor  
        // más uno dividido para dos.  
    }
```

```

    altura = (diagonalMayor + 1) / 2;
}
// Caso contrario,
else
{
    // Asignar a la variable 'altura' el valor de la mitad de la
    // diagonal mayor.
    altura = diagonalMayor / 2;
}

// Bucle externo for()
// Para:
// 1. Inicializar el valor de las filas 'f' en uno.
// 2. Mientras la condición (f <= altura) sea verdadera, ejecutar
// las sentencias del bucle externo.
// 3. Luego de cada ciclo incrementar el valor de las filas 'f' en uno.
for (f = 1; f <= altura; f++)
{
    // Bucle interno for()
    // Para:
    // 1. Inicializar el valor de las columnas de blancos 'b' con
    // el valor de la variable 'altura' menos el número de filas.
    // 2. Mientras la condición (b >= 1) sea verdadera, ejecutar
    // las sentencias del bucle interno.
    // 3. Luego de cada ciclo decrementar el valor de las columnas de
    // blancos 'b' en uno.
    for (b = altura - f; b >= 1; b--)
    {
        // Imprimir un espacio en blanco por columna.
        cout << " ";
    }
    // Bucle interno for()
    // Para:
    // 1. Inicializar el valor de las columnas de asteriscos 'a' en uno.
    // 2. Mientras la condición (a <= 2 * f - 1) sea verdadera, ejecutar
    // las sentencias del bucle interno.
    // 3. Luego de cada ciclo incrementar el valor de las columnas de
    // asteriscos 'a' en uno.
    for (a = 1; a <= 2 * f - 1; a++)
    {
        // Imprimir un asterisco por columna.
        cout << "*";
    }
    // Imprimir un salto de línea (INTRO).
    cout << endl;
}

// Bucle externo for()
// Para:
// 1. Inicializar el valor de las filas 'f' con el valor de la variable
// 'altura' menos uno.
// 2. Mientras la condición (f >= 1) sea verdadera, ejecutar
// las sentencias del bucle externo.
// 3. Luego de cada ciclo decrementar el valor de las filas 'f' en uno.
for (f = altura - 1; f >= 1; f--)
{
    // Bucle interno for()
    // Para:
    // 1. Inicializar el valor de las columnas de blancos 'b' con
    // el valor de la altura menos el número de filas.
    // 2. Mientras la condición (b >= 1) sea verdadera, ejecutar
    // las sentencias del bucle interno.

```

D) Implementación

Para implementar la solución, se debe escribir el algoritmo como un programa de C/C++ que contenga toda la información necesaria para completar la traducción a lenguaje máquina. En la Tabla 4.8.1 se muestra el código del programa en C/C++.

```
// 3. Luego de cada ciclo decrementar el valor de las columnas de
// blancos 'b' en uno.
for (b = altura - f; b >= 1; b--)
{
    cout << " ";
}
// Bucle interno for()
// Para:
// 1. Inicializar el valor de las columnas de asteriscos 'a' en uno.
// 2. Mientras la condición (a <= 2 * f - 1) sea verdadera, ejecutar
// las sentencias del bucle interno.
// 3. Luego de cada ciclo incrementar el valor de las columnas de
// asteriscos 'a' en uno.
for (a = 1; a <= 2 * f - 1; a++)
{
    // Imprimir un asterisco por columna.
    cout << "*";
}
// Imprimir un salto de línea (INTRO).
cout << endl;
}

// Incorporar una pausa en el programa.
system("pause");
return 0;
}
```

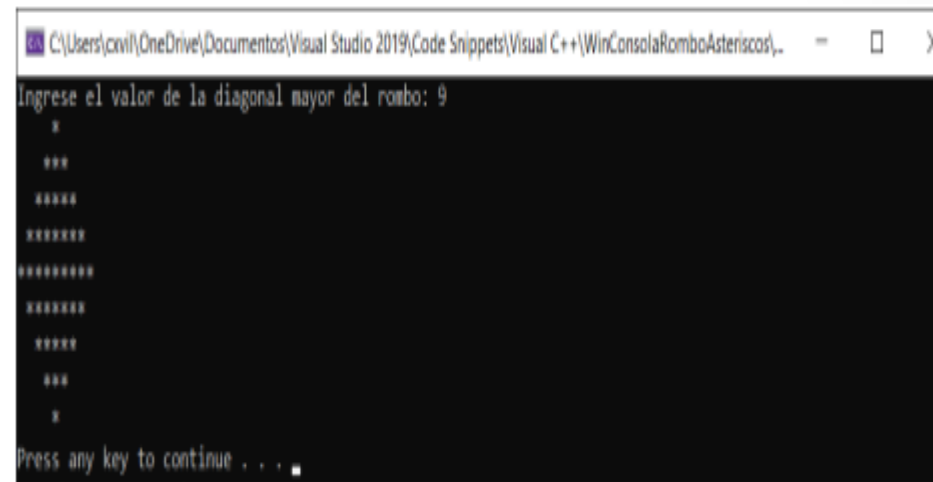
Programa 4.8.1 Código del Programa. Fuente: Zambrano M., et al., 2025.

Caso de Estudio 4.8:

Visualización de un Rombo con Asteriscos 254

E) Pruebas

En la Tabla 4.8.2, se muestra un ejemplo de la salida del programa, donde se puede ver que los resultados del programa tienen sentido, comparando estos resultados con los obtenidos en una hoja de cuadros o de papel milimetrado. En la salida de la Tabla 4.8.2 se puede ver que, al ingresar el valor de la diagonal mayor igual a 9 se obtiene el rombo de asteriscos compuesto por dos triángulos isósceles, el uno de altura igual a 5 y el otro de altura igual a 4.



```
C:\Users\civil\OneDrive\Documentos\Visual Studio 2019\Code Snippets\Visual C++\WinConsoleRomboAsteriscos, - □ X
Ingrese el valor de la diagonal mayor del rombo: 9
 *
 ***
*****
*****
*****
*****
*****
 *
Press any key to continue . . .
```

Ejecución 4.8.2 Ejecución del Programa . Fuente:
Zambrano M., et al., 2025.

Caso de Estudio 4.8:

Visualización de un Rombo

con Asteriscos 254

En la Tabla 4.8.3 se puede ver la primera parte de la prueba de escritorio del programa, donde se almacena en la variable 'diagonalMayor' el valor de 9. Luego se verifica que este valor es impar utilizando el operador de módulo (%). Como este valor es impar, se suma el valor de uno obteniéndose 10 y ese resultado se divide para 2 obteniéndose el valor de 5 en la variable altura, caso contrario, se dividiría directamente para 2.

diagonalMayor	diagonalMayor % 2 != 0	altura = (diagonalMayor + 1) / 2;
9	9%2 != 0 (V)	(9 + 1) / 2 = 10 / 2 = 5

Tabla 4.8.3 Prueba de Escritorio . Fuente: Zambrano M., et al., 2025.

Caso de Estudio 4.8:

Visualización de un Rombo con Asteriscos 254

En la Tabla 4.8.4 se puede ver la segunda parte de la prueba de escritorio del programa, donde la impresión de la mitad del rombo de asteriscos es un triángulo isósceles cuyo valor de altura es 5, se realiza mediante un bucle externo y dos bucles internos. Cada vez que se repite el bucle externo incremental, se ejecutan los dos bucles internos. El bucle externo incremental se repite cinco veces en este caso, según la altura del triángulo y se controla con el contador de las filas 'f'. El número de repeticiones realizadas por los bucles internos se basan en el valor del contador de las filas 'f'. El primer bucle interno controla la impresión de los espacios en blanco con el contador 'b' y el segundo bucle interno controla la impresión de los asteriscos con el contador 'a'.

altura	f	f<=altura	b	b>=1	cout << " ";	a	a<=2*f-1	cout << "***";	cout << endl;
5	1	1<=5(V)	4	4>=1(V)	bbbb	1	1<=1(V)	*	
			3	3>=1(V)		2	2<=1(F)		
			2	2>=1(V)					
			1	1>=1(V)					
			0	0>=1(F)					INTRO
	2	2<=5(V)	3	3>=1(V)	bbb	1	1<=3(V)	***	
			2	2>=1(V)		2	2<=3(V)		
			1	1>=1(V)		3	3<=3(V)		
			0	0>=1(F)		4	4<=3(F)		INTRO
	3	3<=5(V)	2	2>=1(V)	bb	1	1<=5(V)	*****	
			1	1>=1(V)		2	2<=5(V)		
			0	0>=1(F)		3	3<=5(V)		
						4	4<=5(V)		
						5	5<=5(V)		
						6	6<=5(F)		INTRO
	4	4<=5(V)	1	1>=1(V)	b	1	1<=7(V)	*****	
			0	0>=1(F)		2	2<=7(V)		
						3	3<=7(V)		
						4	4<=7(V)		
						5	5<=7(V)		
						6	6<=7(V)		
						7	7<=7(V)		
						8	8<=7(F)		INTRO
	5	5<=5(V)	0	0>=1(F)		1	1<=9(V)	*****	
						2	2<=9(V)		
						3	3<=9(V)		
						4	4<=9(V)		
						5	5<=9(V)		
						6	6<=9(V)		
						7	7<=9(V)		
						8	8<=9(V)		
						9	9<=9(V)		
						10	10<=9(F)		INTRO
	6	6<=5(F)							

Tabla 4.8.4 Prueba de Escritorio . Fuente: Zambrano M., et al., 2025.

Caso de Estudio 4.8:

Visualización de un Rombo

con Asteriscos 254

En la Tabla 4.8.5 se puede ver la tercera parte de la prueba de escritorio del programa, donde la impresión de la mitad del rombo de asteriscos es un triángulo isósceles invertido cuyo valor de altura es 4, se realiza mediante un bucle externo y dos bucles internos. Cada vez que se repite el bucle externo decremental, se ejecutan los dos bucles internos. El bucle externo decremental se repite cuatro veces en este caso, según la altura del triángulo y se controla con el contador de las filas 'f'. El número de repeticiones realizadas por los bucles internos se basan en el valor del contador de las filas 'f'. El primer bucle interno controla la impresión de los espacios en blanco con el contador 'b' y el segundo bucle interno controla la impresión de los asteriscos con el contador 'a'.

altura	f	f>=1	b	b>=1	cout << " ";	a	a<=2*f-1	cout << "***";	cout << endl;
5	4	4>=1(V)	1	1>=1(V)	b	1	1<=7(V)	*****	
			0	0>=1(F)		2	2<=7(V)		
						3	3<=7(V)		
						4	4<=7(V)		
						5	5<=7(V)		
						6	6<=7(V)		
						7	7<=7(V)		
						8	8<=7(F)		INTRO
	3	3>=1(V)	2	2>=1(V)	bb	1	1<=5(V)	*****	
			1	1>=1(V)		2	2<=5(V)		
			0	0>=1(F)		3	3<=5(V)		
						4	4<=5(V)		
						5	5<=5(V)		
						6	6<=5(F)		INTRO
	2	2>=1(V)	3	3>=1(V)	bbb	1	1<=3(V)	***	
			2	2>=1(V)		2	2<=3(V)		
			1	1>=1(V)		3	3<=3(V)		
			0	0>=1(F)		4	4<=3(F)		INTRO
	1	1>=1(V)	4	4>=1(V)	bbbb	1	1<=1(V)	*	
			3	3>=1(V)		2	2<=1(F)		
			2	2>=1(V)					
			1	1>=1(V)					
			0	0>=1(F)					INTRO
	0	0>=1(F)							

Tabla 4.8.5 Prueba de Escritorio . Fuente: Zambrano M., et al., 2025.

Pensamiento

Científico de la Computación

- “Todo el mundo debería saber programar”

Steve Jobs
1955-2011

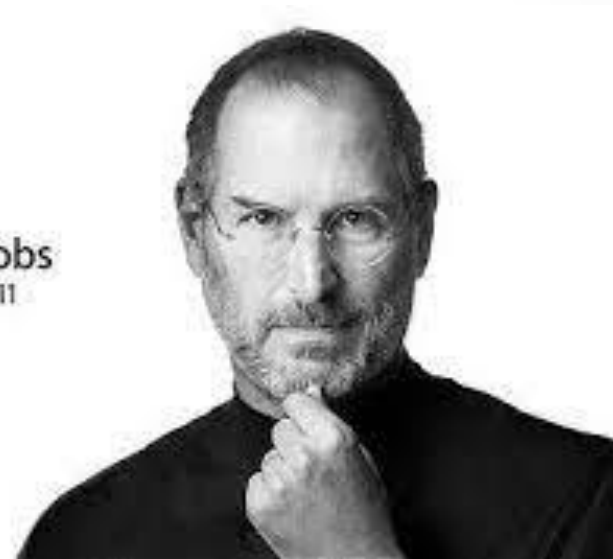


Figura 32. Universidad de las Fuerzas Armadas - ESPE.
Fuente: <https://economipedia.com/definiciones/steve-jobs.html>

¿Preguntas?

!Gracias!